1)Use linked list for polynomial representation and perform addition of subtraction operations on the polynomials.

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int coeff, exp;
    struct Node *next;
};
struct Node *create()
{
    struct Node *head = NULL, *temp, *newnode;
    int n, i;
    printf("Enter number of terms: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        newnode = (struct Node *)malloc(sizeof(struct Node));
        printf("Enter coefficient and exponent: ");
        scanf("%d%d", &newnode->coeff, &newnode->exp);
        newnode->next = NULL;

        if (head == NULL)
            head = newnode;
        else
            temp->next = newnode;

        temp = newnode;
    }
    return head;
}
struct Node *calculate(struct Node *a, struct Node *b, int sign)
{
    struct Node *result = NULL, *temp, *newnode, *p;
    while (a != NULL || b != NULL)
    {
        newnode = (struct Node *)malloc(sizeof(struct Node));
        if (b == NULL || (a != NULL && a->exp > b->exp))
        {
```

```
            newnode->coeff = a->coeff;
            newnode->exp = a->exp;
            a = a->next;
        }
        else if (a == NULL || b->exp > a->exp)
        {
            newnode->coeff = sign * b->coeff;
            newnode->exp = b->exp;
            b = b->next;
        }
        else
        {
            newnode->coeff = a->coeff + sign * b->coeff;
            newnode->exp = a->exp;
            a = a->next;
            b = b->next;
        }
        newnode->next = NULL;
        if (newnode->coeff != 0)
        {
            if (result == NULL)
                result = newnode;
            else
                temp->next = newnode;

            temp = newnode;
        }
        else
            free(newnode);
    }
    return result;
}
void display(struct Node *p)
{
    while (p != NULL)
    {
        printf("%dx^%d", p->coeff, p->exp);
        if (p->next != NULL)
            printf(" + ");
        p = p->next;
```

```
    }
    printf("\n");
}
int main()
{
    struct Node *p1, *p2, *sum, *diff;
    printf("First polynomial:\n");
    p1 = create();
    printf("Second polynomial:\n");
    p2 = create();
    sum = calculate(p1, p2, 1);
    diff = calculate(p1, p2, -1);
    printf("Addition: ");
    display(sum);
    printf("Subtraction: ");
    display(diff);
    return 0;
}
```

OUTPUT :

```
First polynomial:
Enter number of terms: 2
Enter coefficient and exponent: 3 2
Enter coefficient and exponent: 2 1

Second polynomial:
Enter number of terms: 2
Enter coefficient and exponent: 1 2
Enter coefficient and exponent: 4 0

Addition: 4x^2 + 2x^1 + 4x^0
Subtraction: 2x^2 + 2x^1 + -4x^0
```

2) Find previous node of circular LL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
```

```
    int data;
    struct Node *next;
};

int main()
{
    struct Node *head = NULL, *temp, *newnode;
    int n, i, key;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        newnode = (struct Node *)malloc(sizeof(struct Node));

        printf("Enter data: ");
        scanf("%d", &newnode->data);

        if (head == NULL)
        {
            head = newnode;
            newnode->next = head;
        }
        else
        {
            temp = head;
            while (temp->next != head)
                temp = temp->next;

            temp->next = newnode;
            newnode->next = head;
        }
    }

    printf("Enter node value: ");
    scanf("%d", &key);

    temp = head;
```

```
    do
    {
        if (temp->next->data == key)
        {
            printf("Previous node = %d\n", temp->data);
            return 0;
        }
        temp = temp->next;
    } while (temp != head);

    printf("Node not found\n");

    return 0;
}
```

OUTPUT :

```
Enter number of nodes: 4
Enter data: 10
Enter data: 20
Enter data: 30
Enter data: 40
Enter node value: 30
Previous node = 20
```

3) Insert at a given position of a circular LL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

int main()
{
    struct Node *head = NULL, *temp, *newnode;
```

```
int n, i, pos, value;

printf("Enter number of nodes: ");
scanf("%d", &n);

for (i = 0; i < n; i++)
{
   newnode = (struct Node *)malloc(sizeof(struct Node));
   scanf("%d", &newnode->data);

   if (head == NULL)
   {
      head = newnode;
      newnode->next = head;
   }
   else
   {
      temp = head;
      while (temp->next != head)
         temp = temp->next;

      temp->next = newnode;
      newnode->next = head;
   }
}

printf("Enter value and position: ");
scanf("%d%d", &value, &pos);

newnode = (struct Node *)malloc(sizeof(struct Node));
newnode->data = value;

if (pos < 1 || pos > n + 1)
{
   printf("Invalid position\n");
   return 0;
}

if (pos == 1)
{
```

```
        temp = head;
        while (temp->next != head)
            temp = temp->next;

        newnode->next = head;
        temp->next = newnode;
        head = newnode;
    }
    else
    {
        temp = head;

        for (i = 1; i < pos - 1; i++)
            temp = temp->next;

        newnode->next = temp->next;
        temp->next = newnode;
    }

    temp = head;
    do
    {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != head);

    return 0;
}
```

OUTPUT :

```
Enter number of nodes: 4
10 20 30 40
Enter value and position: 25 3
10 20 25 30 40
```

4) Find largest 2nd element in a Queue.

```c
#include <stdio.h>

int main()
{
    int a[100], n, i, j, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter queue elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (a[i] < a[j])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }

    if (n < 2)
        printf("Not enough elements\n");
    else
    {
        for (i = 1; i < n; i++)
        {
            if (a[i] != a[0])
            {
                printf("Second largest element = %d\n", a[i]);
                return 0;
            }
        }

        printf("No second distinct largest element\n");
```

```
    }

    return 0;
}
```

OUTPUT :

```
Enter number of elements: 5
Enter queue elements: 10 40 20 50 30
Second largest element = 40
```

5) Insert 3rd element and check whether 1st and 3rd elements are same in a LL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

int main()
{
    struct Node *head = NULL, *temp, *newnode;
    int n, i, value, first, third;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        newnode = (struct Node *)malloc(sizeof(struct Node));
        scanf("%d", &newnode->data);
        newnode->next = NULL;

        if (head == NULL)
            head = newnode;
```

```
    else
        temp->next = newnode;

    temp = newnode;
}

if (n < 3)
{
    printf("At least 3 nodes are required\n");
    return 0;
}

printf("Enter value to insert at 3rd position: ");
scanf("%d", &value);

newnode = (struct Node *)malloc(sizeof(struct Node));
newnode->data = value;

temp = head;
for (i = 1; i < 2; i++)
    temp = temp->next;

newnode->next = temp->next;
temp->next = newnode;

first = head->data;
third = head->next->next->data;

temp = head;
printf("Linked list: ");
while (temp != NULL)
{
    printf("%d ", temp->data);
    temp = temp->next;
}

if (first == third)
    printf("\nFirst and third elements are same\n");
else
    printf("\nFirst and third elements are different\n");
```

```
    return 0;
}
```

OUTPUT :

```
Enter number of nodes: 3
10 20 30
Enter value to insert at 3rd position: 10
Linked list: 10 20 10 30
First and third elements are same
```

6) Implement two stacks in one array write Routines to traverse the array and display.

```
#include <stdio.h>
#define SIZE 10

int a[SIZE];
int top1 = -1;
int top2 = SIZE;

void push1(int value)
{
    if (top1 + 1 == top2)
        printf("Stack Overflow\n");
    else
        a[++top1] = value;
}

void push2(int value)
{
    if (top1 + 1 == top2)
        printf("Stack Overflow\n");
    else
        a[--top2] = value;
}

void display1()
{
```

```
    int i;

    printf("Stack 1: ");
    for (i = top1; i >= 0; i--)
        printf("%d ", a[i]);

    printf("\n");
}

void display2()
{
    int i;

    printf("Stack 2: ");
    for (i = top2; i < SIZE; i++)
        printf("%d ", a[i]);

    printf("\n");
}

int main()
{
    push1(10);
    push1(20);
    push1(30);

    push2(40);
    push2(50);
    push2(60);

    display1();
    display2();

    return 0;
}
```

OUTPUT :

```
Stack 1: 30 20 10
Stack 2: 60 50 40
```

7) Given a linked list containing n head nodes where every node in the LL contains two pointers — one pointer to next node and bottom pointer to a sub-linked list where the current node is the head of the sub-linked list. List nodes and all head nodes are sorted in ascending order based on their data. We need to flatten the LL so that all nodes appear in a single LL while maintaining sorted order.

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
    struct Node *bottom;
};

struct Node *createNode(int data)
{
    struct Node *newnode;

    newnode = (struct Node *)malloc(sizeof(struct Node));
    newnode->data = data;
    newnode->next = NULL;
    newnode->bottom = NULL;

    return newnode;
}

struct Node *merge(struct Node *a, struct Node *b)
{
    struct Node dummy;
    struct Node *tail = &dummy;

    dummy.bottom = NULL;

    while (a != NULL && b != NULL)
    {
        if (a->data <= b->data)
        {
            tail->bottom = a;
```

```
            a = a->bottom;
        }
        else
        {
            tail->bottom = b;
            b = b->bottom;
        }

        tail = tail->bottom;
    }

    if (a != NULL)
        tail->bottom = a;
    else
        tail->bottom = b;

    return dummy.bottom;
}

struct Node *flatten(struct Node *head)
{
    struct Node *result;

    if (head == NULL || head->next == NULL)
        return head;

    result = flatten(head->next);

    head->next = NULL;

    return merge(head, result);
}

void display(struct Node *head)
{
    while (head != NULL)
    {
        printf("%d ", head->data);
        head = head->bottom;
    }
```

```
    printf("\n");
}

int main()
{
    struct Node *head, *n2, *n3, *n4;

    head = createNode(5);
    n2 = createNode(10);
    n3 = createNode(19);
    n4 = createNode(28);

    head->next = n2;
    n2->next = n3;
    n3->next = n4;

    head->bottom = createNode(7);
    head->bottom->bottom = createNode(8);

    n2->bottom = createNode(20);

    n3->bottom = createNode(22);

    n4->bottom = createNode(35);
    n4->bottom->bottom = createNode(40);
    n4->bottom->bottom->bottom = createNode(45);

    head = flatten(head);

    printf("Flattened linked list:\n");
    display(head);

    return 0;
}
```

OUTPUT :

```
Flattened linked list:
5 7 8 10 19 20 22 28 35 40 45
```

8) Given K sorted LL of different sizes, we need to merge them into a single list maintaining their sorted order.

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node *createNode(int data)
{
    struct Node *newnode;

    newnode = (struct Node *)malloc(sizeof(struct Node));
    newnode->data = data;
    newnode->next = NULL;

    return newnode;
}

struct Node *merge(struct Node *a, struct Node *b)
{
    struct Node dummy;
    struct Node *tail = &dummy;

    dummy.next = NULL;

    while (a != NULL && b != NULL)
    {
        if (a->data <= b->data)
        {
            tail->next = a;
            a = a->next;
        }
        else
        {
            tail->next = b;
```

```
            b = b->next;
        }

        tail = tail->next;
    }

    if (a != NULL)
        tail->next = a;
    else
        tail->next = b;

    return dummy.next;
}

void display(struct Node *head)
{
    while (head != NULL)
    {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main()
{
    struct Node *a, *b, *c, *result;

    a = createNode(1);
    a->next = createNode(4);
    a->next->next = createNode(7);

    b = createNode(2);
    b->next = createNode(5);

    c = createNode(3);
    c->next = createNode(6);
    c->next->next = createNode(8);

    result = merge(a, b);
```

```
    result = merge(result, c);

    printf("Merged linked list:\n");
    display(result);

    return 0;
}
```

OUTPUT :

```
Merged linked list:
1 2 3 4 5 6 7 8
```

9) Swap kth node from beginning with kth node from end in a LL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

int main()
{
    struct Node *head = NULL, *temp, *newnode;
    struct Node *x, *y, *prevX, *prevY, *t;
    int n, k, i, posX, posY;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        newnode = (struct Node *)malloc(sizeof(struct Node));
        scanf("%d", &newnode->data);
        newnode->next = NULL;
```

```
    if (head == NULL)
        head = newnode;
    else
        temp->next = newnode;

    temp = newnode;
}

printf("Enter k: ");
scanf("%d", &k);

if (k < 1 || k > n)
{
    printf("Invalid k\n");
    return 0;
}

posX = k;
posY = n - k + 1;

if (posX == posY)
{
    printf("No swap needed\n");
}
else
{
    if (posX > posY)
    {
        i = posX;
        posX = posY;
        posY = i;
    }

    prevX = NULL;
    x = head;

    for (i = 1; i < posX; i++)
    {
        prevX = x;
        x = x->next;
```

```
    }

    prevY = NULL;
    y = head;

    for (i = 1; i < posY; i++)
    {
        prevY = y;
        y = y->next;
    }

    if (x->next == y)
    {
        if (prevX != NULL)
            prevX->next = y;
        else
            head = y;

        x->next = y->next;
        y->next = x;
    }
    else
    {
        if (prevX != NULL)
            prevX->next = y;
        else
            head = y;

        prevY->next = x;

        t = x->next;
        x->next = y->next;
        y->next = t;
    }
}

printf("After swapping: ");

temp = head;
while (temp != NULL)
```

```
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }

    return 0;
}
```

OUTPUT :

```
Enter number of nodes: 5
10 20 30 40 50
Enter k: 2
After swapping: 10 40 30 20 50
```

10) Multiply two numbers represented by two LL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node *createNode(int data)
{
    struct Node *newnode;

    newnode = (struct Node *)malloc(sizeof(struct Node));
    newnode->data = data;
    newnode->next = NULL;

    return newnode;
}

struct Node *createList(int n)
{
    struct Node *head = NULL, *temp, *newnode;
```

```
    int i, digit;

    for (i = 0; i < n; i++)
    {
        printf("Enter digit: ");
        scanf("%d", &digit);

        newnode = createNode(digit);

        if (head == NULL)
            head = newnode;
        else
            temp->next = newnode;

        temp = newnode;
    }

    return head;
}

long long convert(struct Node *head)
{
    long long num = 0;

    while (head != NULL)
    {
        num = num * 10 + head->data;
        head = head->next;
    }

    return num;
}

int main()
{
    struct Node *a, *b;
    int n1, n2;
    long long num1, num2, product;

    printf("Enter number of digits in first number: ");
```

```
    scanf("%d", &n1);
    a = createList(n1);

    printf("Enter number of digits in second number: ");
    scanf("%d", &n2);
    b = createList(n2);

    num1 = convert(a);
    num2 = convert(b);

    product = num1 * num2;

    printf("Product = %lld\n", product);

    return 0;
}
```

OUTPUT :

```
Enter number of digits in first number: 3
Enter digit: 1
Enter digit: 0
Enter digit: 0
Enter number of digits in second number: 2
Enter digit: 1
Enter digit: 0
Product = 1000
```